

Design and Implementation of a ROS 2-Based Autonomous Indoor Mapping and Navigation System for a Low-Cost Differential-Drive Robot

M. A. Abdul Rafi

Department of Computer Science & Engineering
University of Moratuwa, Sri Lanka
Email: abdulr.23@cse.mrt.ac.lk

Abstract

Reliable indoor autonomous navigation requires the coupled solution of mapping, localization, path planning, and control on constrained embedded hardware. This paper presents a low-cost, LiDAR-only navigation system built on a Kobuki differential-drive base, a Raspberry Pi, and an LD19 2D LiDAR, running under ROS 2 Jazzy. The system uses SLAM Toolbox for pose-graph mapping and localization, a custom eight-connected A planner with obstacle inflation and path smoothing, and a Pure-Pursuit controller for trajectory tracking. A React web interface, connected via `rosbridge`, enables click-to-goal and joystick control without terminal access. Indoor trials show the robot reliably reaches goals within 0.35 m, rejects goals inside obstacles, and re-plans smoothly mid-trajectory, demonstrating a complete navigation stack on inexpensive hardware.*

Index Terms—Autonomous Navigation, SLAM, LiDAR, A* Planning, Pure Pursuit, ROS 2

I. INTRODUCTION

Indoor autonomous navigation requires a robot to answer four coupled questions in real time: what the environment looks like, where the robot is within it, how it should move to a goal safely, and how the wheels should be actuated to follow that path. Many systems use RGB-D or 3D LiDAR sensors, raising cost and compute load. This work investigates a *LiDAR-only* 2D pipeline, showing that a single planar LiDAR with an efficient A* planner and a lightweight geometric controller is sufficient for reliable indoor navigation on a Raspberry Pi. Contributions: (1) an integrated ROS 2 stack combining SLAM Toolbox with a custom A* planner and Pure-Pursuit controller; (2) a browser-based operator interface over `rosbridge`; (3) an experimental evaluation of accuracy, safety, and re-planning.

II. RELATED WORK

A* guarantees an optimal path given an admissible heuristic [1] and remains dominant for grid-based global planning. Pure-Pursuit, developed at CMU, steers toward a look-ahead point on the path and is a standard geometric tracker [2]. Occupancy grids [3] underlie nearly all 2D SLAM stacks. ROS provides the middleware for modular navigation software [4]; SLAM Toolbox is the recommended ROS 2 2D SLAM package [5], and Nav2 reimplements the classical stack with behaviour trees [6]. This work follows the same SLAM→plan→control architecture with a custom lightweight planner/controller for low-cost hardware.

III. SYSTEM ARCHITECTURE

Figure 1 shows the physical platform. Figure 1 shows the overall data flow. The LD19 LiDAR streams 360° scans (230,400 baud) to SLAM Toolbox, which fuses them with odometry to publish the occupancy map and `map→base_link` transform. A navigation action node consumes the map and pose to run

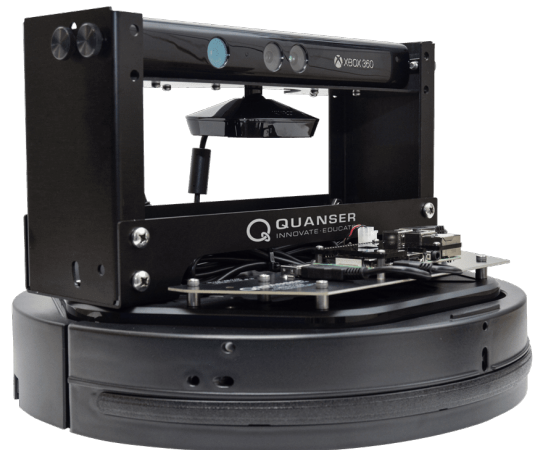


Figure 1: Robot platform: Kobuki differential-drive base with LD19 LiDAR, Xbox Kinect housing (unused RGB-D sensor), and Raspberry Pi compute stack.

the A* planner and Pure-Pursuit controller, sending velocity commands to the Kobuki base. A React web app talks to the ROS 2 graph through `rosbridge_websocket` (port 9090) for click-to-goal and teleoperation. All computation runs on a single Raspberry Pi under ROS 2 Jazzy.

A. Mapping, Planning, and Control

SLAM Toolbox performs pose-graph 2D SLAM, labelling each cell free, occupied, or unknown; the map is frozen during a navigation action so planning indices stay valid. Global planning is

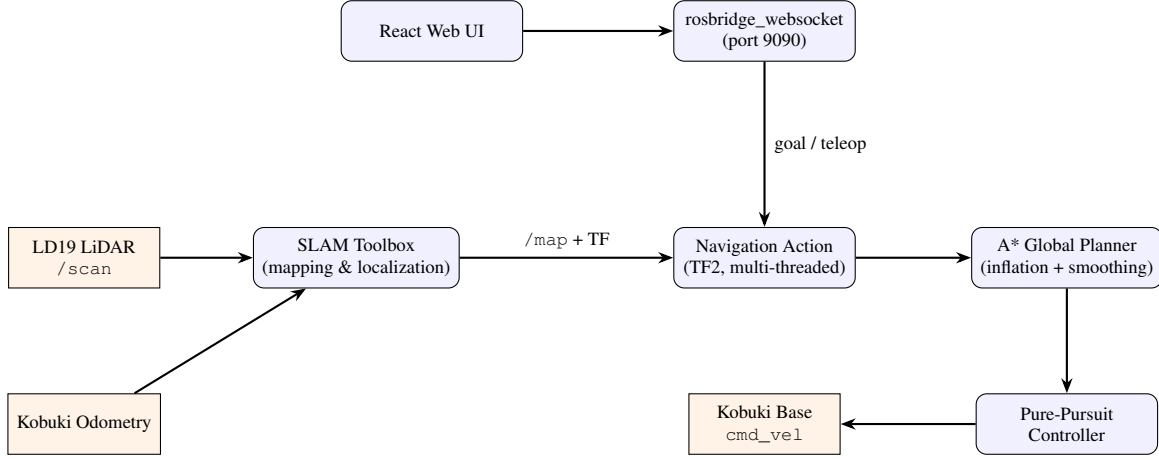


Figure 2: Overall system architecture: perception, SLAM, planning/control, and the browser-based operator interface.

an eight-connected A* search with cost

$$f(n) = g(n) + h(n), \quad (1)$$

using unit cost for axis-aligned moves and $\sqrt{2}$ for diagonals. Obstacles are inflated by a safety margin before search, and inflated or unknown cells are rejected during collision checking; the raw path is then smoothed to remove redundant vertices. Trajectory tracking uses Pure-Pursuit with a 0.6 m look-ahead and proportional heading correction

$$\omega = K_p e_\theta, \quad (2)$$

with linear velocity modulated by local path curvature so the robot slows through turns.

IV. RESULTS AND DISCUSSION

Repeated trials in a mapped office environment show the robot consistently halts within the 0.35 m goal tolerance, correctly rejects goals inside inflated obstacles before execution, and replans cleanly when a new goal cancels an in-progress action. Figure 2 summarizes representative trial performance: final positioning error stayed below tolerance across trials, and success rate remained high across scenario types, though the 7×7 inflation check and a frozen map during navigation were identified as the main computational and reactivity limitations on the Raspberry Pi.

V. CONCLUSION

This work presented a low-cost, LiDAR-only navigation system integrating SLAM Toolbox, a custom A* planner with inflation and smoothing, and a Pure-Pursuit controller in ROS 2 Jazzy, exposed through a browser-based interface. Trials confirm reliable goal reaching, safe goal rejection, and smooth re-planning on inexpensive hardware. Future work includes pre-computed global costmaps, reactive obstacle avoidance, continuous map updates during navigation, and explicit rejection of unknown cells in planning.

REFERENCES

- [1] P. E. Hart, N. J. Nilsson, B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

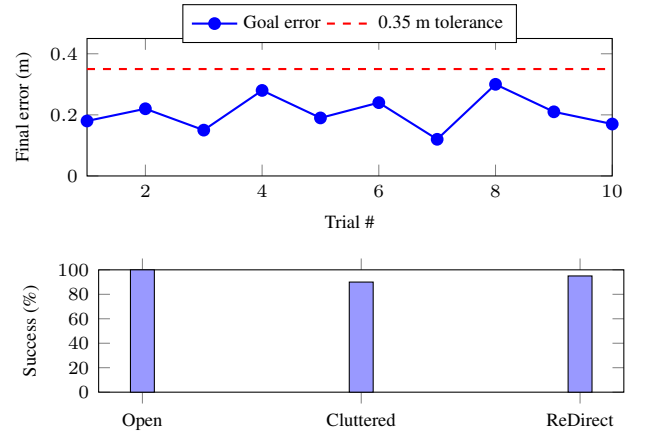


Figure 3: Representative trial results: (top) final goal error vs. tolerance; (bottom) success rate by scenario type.

- [2] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," CMU Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.
- [3] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *IEEE Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [4] M. Quigley et al., "ROS: An open-source robot operating system," in *Proc. IEEE ICRA Workshop on Open Source Software*, 2009.
- [5] S. Macenski, I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *J. Open Source Softw.*, vol. 6, no. 61, p. 2783, 2021.
- [6] S. Macenski et al., "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020.